Supplementary Materials

Generalized Mean Reflected Mass for Subspace-Aware Robot Motion and Impact Evaluation

Project website Code repository

Contents

S1. Simulation controller 1
S2. Simulation settings and execution 2
S3. Experiment 3: Settings and joint configurations 3
S4. Online hardware validation 5
Appendix A. Simulation controller source 6

S1. Simulation controller

The videos demonstrate online QP control of a simulated Franka Panda in Robotics Toolbox for Python and Swift. The controller uses arm redundancy to increase or decrease the reflected mass while tracking an end-effector pose target. The simulation uses a fixed step of $\Delta t = 0.025$ s, corresponding to a nominal update rate of 40 Hz.

Controller demonstrations

Video S1 — Minimizing the X–Z GMRM

Video S2 — Maximizing the X–Z GMRM

The QP is solved at each simulation step; wall-clock timing statistics are not reported.

S1.1. QP formulation

The reflected-mass metric enters a QP as a secondary objective, following the optimization and motion-control formulations of Nocedal and Wright (2006) and Haviland and Corke (2021):

$$\min_{\boldsymbol{x}} \quad \frac{1}{2}\boldsymbol{x}^T\boldsymbol{Q}\boldsymbol{x} + \boldsymbol{C}^T\boldsymbol{x},$$

$$\text{subject to} \quad \mathcal{J}\boldsymbol{x} = \boldsymbol{\nu}^*, \quad \boldsymbol{X}^- \leq \boldsymbol{x} \leq \boldsymbol{X}^+.$$

For n joints, the decision variable consists of joint velocities and tracking slack:

$$\boldsymbol{x} = \begin{pmatrix} \dot{\boldsymbol{q}} \\ \boldsymbol{\delta} \end{pmatrix} \in \mathbb{R}^{n+6}, \quad \boldsymbol{Q} = \begin{pmatrix} \lambda_q I_n & 0 \\ 0 & \lambda_\delta I_6 \end{pmatrix},$$

$$\mathcal{J} = (\boldsymbol{J} \;\; I_6), \quad \boldsymbol{C} = \begin{pmatrix} s\alpha\nabla m(\boldsymbol{q}) \\ 0_{6\times 1} \end{pmatrix}, \quad \boldsymbol{X}^\pm = \begin{pmatrix} \dot{\boldsymbol{q}}^\pm \\ \boldsymbol{\delta}^\pm \end{pmatrix}.$$

Here $\boldsymbol{\delta} \in \mathbb{R}^6$ relaxes twist tracking; λ_q and λ_δ weight joint velocity and slack. The desired twist $\boldsymbol{\nu}^*$ and Jacobian $\boldsymbol{J}$ are expressed in the end-effector frame in this implementation. The metric is $m(\boldsymbol{q}) = \overline{m}^{(2)}_{\mathcal{S}_{\mathrm{XZ}}}$, evaluated in the base-frame X–Z plane. The linear term favors decreasing the metric for $s = +1$ and increasing it for $s = -1$, with $\alpha > 0$. The bounds constrain joint velocity and slack.

S2. Simulation settings and execution

These settings describe the supplied simulation example. The included source defaults to minimization; set `gmrm_gain = -0.2` to run the maximization variant with the other settings unchanged.

Setting	Value / implementation
Robot and initial configuration	`rtb.models.URDF.Panda()`; `panda.qr`
Evaluation plane	Base-frame X–Z; basis $\begin{pmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{pmatrix}$
Target pose	Initial orientation retained; base-frame position offset $(0.1, -0.3, -0.25)$ m
Simulation step	0.025 s; at most 2,000 control steps
Metric coefficient $s\alpha$	+0.2 (minimize); −0.2 (maximize)
Joint-velocity weight λ_q	0.01
Slack weight λ_δ	$\frac{1}{e_t}$; $e_t = \max\left(\sum_{i=1}^{3} \lvert p_{\mathrm{err},i} \rvert, 10^{-6}\right)$
Pose-servo gain	2.0
Bounds	Model joint-velocity limits; slack components ± 10
Termination	$e_t < 0.02$; translational error in the end-effector frame
Gradient	Central differences; joint perturbation 10^{-4} rad
QP solver	`quadprog` if available; otherwise `osqp`

The implementation evaluates the inverse translational inertia as $\boldsymbol{A}_v = \boldsymbol{J}_v \boldsymbol{M}^{-1} \boldsymbol{J}_v^T$ and restricts it to the selected plane. A smooth eigenvalue floor with `EPS = 1e-9` regularizes the numerical metric near singularity. The reported rank is diagnostic; it does not change the averaging dimension in this demo. If the solver returns no solution, the commanded joint velocity is zero.

S2.1. Getting started

Install uv, then run:

```
git clone https://github.com/haoueei/gmrm-code.git
cd gmrm-code
git checkout 1095e1201eb785fcbc57215036769cfad4a4ff3b
uv sync --locked
uv run python main.py
```

The repository revision above fixes the source version used for this document. A copy of `main.py` is supplied as `controller_demo.py` and reproduced in Appendix A.

S2.2. References

J. Nocedal and S. J. Wright, *Numerical Optimization*. Springer, 2006.

J. Haviland and P. Corke, "NEO: A Novel Expeditious Optimisation Algorithm for Reactive Motion Control of Manipulators," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1043–1050, 2021.

S3. Experiment 3: Settings and joint configurations

This section records the parameter settings for Experiment 3 in the manuscript. These are distinct from the simulation-demo settings in Section S2.

PRMS setup

Item	Setting
Spring stiffness	150 N/mm
Contact pad	Black silicone, 7 mm thick, Shore A 70
Body-region setting	Skull and forehead
Mounting	The PRMS was bolted to the table.
Repetitions	10 trials per GMRM objective–speed condition (min/max × 0.3/0.5 m/s), totaling 40 collision trials.

S3.1. QP parameters

The same parameter rules are used across metric objectives within each motion phase. Phase (a) moves the end-effector toward the pre-collision pose; phase (b) tracks a prescribed constant twist toward impact.

Parameter	Phase (a)	Phase (b)
λ_q	0.01	0.001
λ_δ	$\frac{1}{e_{\text{pose}}}$	$\frac{1}{e_{\text{pose}}}$
α	0.55	0.001
β	20	—

Here e_{pose} denotes the total pose error. The rule $\lambda_\delta = \frac{1}{e_{\text{pose}}}$ is fixed, while its numerical value varies with the error. The dash indicates that the pose-servo gain β is not used in phase (b), where the desired twist is prescribed directly.

S3.2. Built-in collision thresholds

The following configuration is applied before the impact motion. Comments identify the eight argument groups; `accel` denotes acceleration/deceleration and `nominal` denotes constant-velocity motion.

```
robot.setCollisionBehavior(
    // Joint contact detection (accel)
    {{100.0, 100.0, 100.0, 100.0, 100.0, 100.0, 100.0}},
    // Joint collision reflex (accel)
    {{100.0, 100.0, 100.0, 100.0, 100.0, 100.0, 100.0}},
    // Joint contact detection (nominal)
    {{100.0, 100.0, 100.0, 100.0, 100.0, 100.0, 100.0}},
    // Joint collision reflex (nominal)
    {{100.0, 100.0, 100.0, 100.0, 100.0, 100.0, 100.0}},
    // Cartesian contact (accel)
    {{100.0, 100.0, 100.0, 100.0, 100.0, 100.0}},
    // Cartesian collision (accel)
    {{100.0, 100.0, 100.0, 100.0, 100.0, 100.0}},
    // Cartesian contact (nominal)
    {{100.0, 100.0, 100.0, 100.0, 100.0, 100.0}},
    // Cartesian collision (nominal)
    {{100.0, 100.0, 100.0, 100.0, 100.0, 100.0}});
```

The first four arrays specify per-joint torque thresholds in Nm. In each Cartesian array, the first three entries are force thresholds in N and the last three are moment thresholds in Nm. Contact and collision thresholds are equal, and the same values are used in both motion phases of the detection interface.

S3.3. Estimated approach velocities

We used the robot Jacobian to estimate approach velocities from the recorded joint states. The table reports velocities along the impact direction for the manuscript's GMRM collision-force trials. Values are means ± across-trial sample standard deviations ($n = 10$ per condition).

Objective	**Commanded speed** (m/s)	**Estimated approach speed** (m/s)
GMRM min	0.3	0.2985 ± 0.0011
GMRM max	0.3	0.3018 ± 0.0013
GMRM min	0.5	0.4927 ± 0.0019
GMRM max	0.5	0.5096 ± 0.0021

S3.4. Joint configurations for Table II

The tables below list the joint configurations used to evaluate the metrics at the pre-collision and collision configurations in Table II. All joint angles are in radians.

Pre-collision configurations at ${}^0\boldsymbol{T}_{e^{\mathrm{pre}}}$

Metric	**Optimization**	$\lvert\boldsymbol{v}_c\rvert$ (m/s)	q_1	q_2	q_3	q_4	q_5	q_6	q_7
$\overline{m}^{(2)}_{\mathcal{S}_{\mathrm{XZ}}}$	min	0.3	−2.415993	−1.6628	1.042458	−1.355408	1.795963	1.080474	−0.54838
$\overline{m}^{(2)}_{\mathcal{S}_{\mathrm{XZ}}}$	max	0.3	−0.344666	0.544538	0.450972	−1.42681	−0.242321	1.916174	0.131551
$\overline{m}^{(2)}_{\mathcal{S}_{\mathrm{XZ}}}$	min	0.5	−2.415667	−1.6628	1.043048	−1.355672	1.795648	1.081026	−0.548479
$\overline{m}^{(2)}_{\mathcal{S}_{\mathrm{XZ}}}$	max	0.5	−0.258995	0.525942	0.336305	−1.429491	−0.177607	1.925492	0.096884
$\overline{m}_u$	min	0.3	−2.418303	−1.6628	1.039606	−1.352111	1.798645	1.0783	−0.546315
$\overline{m}_u$	max	0.3	2.313591	−0.961566	−1.822886	−1.385769	−0.923679	1.666597	0.446008
$\overline{m}_u$	min	0.5	−2.417846	−1.6628	1.040316	−1.352455	1.798269	1.079117	−0.546388
$\overline{m}_u$	max	0.5	2.312723	−0.964521	−1.819769	−1.385898	−0.927288	1.66516	0.447375

Collision configurations at ${}^0\boldsymbol{T}_{e^{\mathrm{col}}}$

Metric	**Optimization**	$\lvert\boldsymbol{v}_c\rvert$ (m/s)	q_1	q_2	q_3	q_4	q_5	q_6	q_7
$\overline{m}^{(2)}_{\mathcal{S}_{\mathrm{XZ}}}$	min	0.3	−2.259462	−1.662677	1.422477	−1.485477	1.679532	1.454526	−0.59087
$\overline{m}^{(2)}_{\mathcal{S}_{\mathrm{XZ}}}$	max	0.3	−0.329165	0.712291	0.405711	−1.549099	−0.325934	2.195196	0.182139
$\overline{m}^{(2)}_{\mathcal{S}_{\mathrm{XZ}}}$	min	0.5	−2.266126	−1.662766	1.436176	−1.473182	1.684093	1.499637	−0.589759
$\overline{m}^{(2)}_{\mathcal{S}_{\mathrm{XZ}}}$	max	0.5	−0.252648	0.68843	0.308799	−1.543724	−0.246582	2.195475	0.139019
$\overline{m}_u$	min	0.3	−2.259301	−1.66277	1.43362	−1.482072	1.678405	1.465696	−0.588601
$\overline{m}_u$	max	0.3	2.307219	−1.144174	−1.918429	−1.495866	−1.104438	1.853253	0.524402
$\overline{m}_u$	min	0.5	−2.26856	−1.662785	1.435818	−1.468658	1.685085	1.500125	−0.587345
$\overline{m}_u$	max	0.5	2.313533	−1.13575	−1.935741	−1.488735	−1.09868	1.863881	0.523655

Model: `franka_panda_no_gripper_ee_inertial_no.urdf`; frame: `panda_ee`.

S4. Online hardware validation

We tested online minimization and maximization of the base-frame X–Z GMRM on a Franka Panda. Each objective was tested twice using the same target pose and controller settings; only the sign of the metric objective changed. These motion trials are separate from the collision experiments in Section S3.

S4.1. Experimental settings

The C++ controller ran on an Intel Core i7-13850HX computer with Ubuntu 20.04.6, using libfranka 0.9.2 for robot-model evaluation and control and ProxQP (ProxSuite 0.7.2) for optimization. A separate thread solves the QP from the latest robot state at a scheduled rate of 50 Hz. The libfranka control callback executes the latest joint-velocity command with SDK filtering and rate limiting.

Setting	**Value**
Nominal initial configuration	$(0, -0.3, 0, -2.2, 0, 2.0, \frac{\pi}{4})$ rad
Target pose	Base-frame offset $(0.225, 0, -0.05)$ m from the nominal initial pose; orientation retained
QP update period	20 ms (50 Hz)
Joint-velocity weight λ_q	0.01
Slack weight λ_δ	$\frac{1}{e_t}$, as defined in Section S2
Metric coefficient $s\alpha$	+0.2 (minimize); −0.2 (maximize)
Pose-servo gain	2.0
Slack bounds	± 10 per component
Joint-velocity limits	50% of the nominal joint-velocity limits
QP joint-position margin	0.02 rad inside the motion envelope
Arrival criteria	Position error < 2 mm and orientation error < 0.005 rad

S4.2. Computation time

The table summarizes the logged QP updates for the four displayed trials. Timing includes model evaluation, reflected-mass and gradient computation, and the QP solve. Across all 565 updates, the mean was 0.231 ms and the maximum was 0.455 ms.

Objective	**Trial**	**Updates**	**Mean** (ms)	**Maximum** (ms)
Minimize	01	132	0.230	0.392
Minimize	02	132	0.238	0.446
Maximize	01	150	0.227	0.455
Maximize	02	151	0.230	0.392

Recordings of all four trials and synchronized plots of GMRM, position error, computation time, and joint velocities are available under Online Hardware Validation on the supplementary website.

Appendix A. Simulation controller source

Source snapshot: `gmrm-code/main.py`, revision `1095e12`. Part 1 of 3.

```
import numpy as np
import qpsolvers as qp
import roboticstoolbox as rtb
import spatialgeometry as sg
import swift

QP_SOLVER = "quadprog" if "quadprog" in qp.available_solvers else "osqp"
EPS = 1e-9
XZ_BASIS = np.array([[1.0, 0.0], [0.0, 0.0], [0.0, 1.0]])


def translational_lambda_inv(robot, q):
    Jv = np.asarray(robot.jacob0(q), dtype=float)[:3]
    M = np.asarray(robot.inertia(q), dtype=float)
    return Jv @ np.linalg.solve(M, Jv.T)


def restricted_eigvals(Av, B0=None):
    B0 = np.eye(3) if B0 is None else np.asarray(B0, dtype=float)
    A0 = B0.T @ np.asarray(Av, dtype=float) @ B0
    return np.linalg.eigvalsh((A0 + A0.T) / 2.0)


def hard_rank_from_av(Av, B0=None, eps=EPS):
    return int(np.count_nonzero(restricted_eigvals(Av, B0) > eps))


def gmrm_from_av(Av, B0=None, eps=EPS):
    sigma = restricted_eigvals(Av, B0)
    width = eps * 0.1
    # Smooth floor avoids discrete rank drops when an eigenvalue crosses eps.
    sigma = eps + width * np.logaddexp(0.0, (sigma - eps) / width)
    lambdas = 1.0 / sigma

    if lambdas.size == 1:
        return float(lambdas[0])
    if lambdas.size == 2:
        return float(np.sqrt(lambdas[0] * lambdas[1]))
    if lambdas.size == 3:
        return mean_reflected_mass(
            np.sqrt(lambdas[0]), np.sqrt(lambdas[1]), np.sqrt(lambdas[2])
        )

    raise ValueError("no positive translational modes in the candidate subspace")


def mean_reflected_mass(a, b, c, n=200):
    lambdas = np.linspace(0, np.pi / 2, n)
    gammas = np.linspace(0, np.pi / 2, n)
    L, G = np.meshgrid(lambdas, gammas)
    r = (a * b * c) / np.sqrt(
        c**2 * (b**2 * np.cos(L) ** 2 + a**2 * np.sin(L) ** 2) * np.cos(G) ** 2
        + a**2 * b**2 * np.sin(G) ** 2
    )
    return float(
        (2 / np.pi) * np.trapezoid(np.trapezoid(r**2 * np.cos(G), L[0]), G[:, 0])
    )
```

Appendix A. Simulation controller source (continued)

Part 2 of 3.

```
def gmrm(robot, q, B0=XZ_BASIS):
    return gmrm_from_av(translational_lambda_inv(robot, q), B0)


def gmrm_grad(robot, q, B0=XZ_BASIS):
    h = 1e-4
    grad = np.zeros_like(q)

    for i in range(q.size):
        dq = np.zeros_like(q)
        dq[i] = h
        grad[i] = (gmrm(robot, q + dq, B0) - gmrm(robot, q - dq, B0)) / (2.0 * h)

    return grad


def step_robot(robot, Tep):
    gmrm_gain = 0.2
    servo_gain = 2.0
    slack_limit = 10.0
    arrived_tol = 0.02

    Te = robot.fkine(robot.q).A
    eTep = np.linalg.inv(Te) @ Tep
    et = max(float(np.sum(np.abs(eTep[:3, 3]))), 1e-6)

    Q = np.eye(robot.n + 6)
    Q[: robot.n, : robot.n] *= 0.01
    Q[robot.n :, robot.n :] *= 1.0 / et

    v, _ = rtb.p_servo(Te, Tep, servo_gain)

    Aeq = np.c_[robot.jacobe(robot.q), np.eye(6)]
    beq = v.reshape(6)

    c = np.zeros(robot.n + 6)
    c[: robot.n] = gmrm_gain * gmrm_grad(robot, robot.q)

    lb = -np.r_[robot.qdlim[: robot.n], slack_limit * np.ones(6)]
    ub = np.r_[robot.qdlim[: robot.n], slack_limit * np.ones(6)]

    x = qp.solve_qp(Q, c, A=Aeq, b=beq, lb=lb, ub=ub, solver=QP_SOLVER)
    qd = np.zeros(robot.n) if x is None else x[: robot.n]

    Av = translational_lambda_inv(robot, robot.q)
    return (
        et < arrived_tol,
        qd,
        et,
        gmrm_from_av(Av, XZ_BASIS),
        hard_rank_from_av(Av, XZ_BASIS),
    )
```

Appendix A. Simulation controller source (continued)

Part 3 of 3.

```
def _self_check():
    assert np.isclose(
        gmrm_from_av(np.diag([0.25, 1.0 / 9.0, 0.0]), np.eye(3)[:, :2]), 6.0
    )
    rank_limited = gmrm_from_av(np.diag([0.25, 0.0, 0.0]), np.eye(3)[:, :2])
    assert np.isfinite(rank_limited) and rank_limited > 4.0
    assert hard_rank_from_av(np.diag([0.25, 0.0, 0.0]), np.eye(3)[:, :2]) == 1
    assert np.isclose(
        gmrm_from_av(np.diag([0.25, 1.0 / 9.0, 1.0 / 16.0]), XZ_BASIS), 8.0
    )


_self_check()

env = swift.Swift()
env.launch(realtime=True)

ax_goal = sg.Axes(0.1)
env.add(ax_goal)

panda = rtb.models.URDF.Panda()
panda.q = panda.qr
env.add(panda)

Tep = panda.fkine(panda.q).A
Tep[:3, 3] += np.array([0.1, -0.3, -0.25])
ax_goal.T = Tep

env.set_camera_pose([1, -1, 1.4], [0.4, 0.0, 0.4])

dt = 0.025
for _ in range(10):
    env.step(dt)

last_rank = None
for i in range(2000):
    arrived, panda.qd, et, measure, rank = step_robot(panda, Tep)
    env.step(dt)

    rank_changed = rank != last_rank
    if i % 25 == 0 or arrived or rank_changed:
        print(f"step={i:04d}, et={et:.4f}, gmrm_xz={measure:.4f}, rank_xz={rank}")
    last_rank = rank

    if arrived:
        break

env.hold()
```